

Arithmetic & Logical Operation Programs

5) Write an ALP to add the byte in the RAM at 34h and 35h, store the result in the register R5(LSB) and R6(MSB), using Indirect Addressing Mode.

```
ORG 0000H           ; Initialize the memory
MOV R0,#34H          ; Register R0 loaded with first data location 34h
MOV R6, #00H         ; Initialize carry=0
MOV A,@R0            ; Read first number into accumulator
INC R0               ; Increment R0 (34H+1=35H)
MOV B,@R0            ; Read second number into register B
ADD A,B              ; Addition of two numbers (A=A+B)
JNC SKIP             ; Jump to label skip if no carry generated
INC R6               ; If addition results in carry, increment carry R6=R6+1
SKIP: MOV R5, A       ; Stores the sum value into register R5
END                  ; End of the program
```

6) Write an ALP to subtract the bytes in the internal RAM 34h and 35h, store the result in the register R5(LSB) and R6(MSB).

```
ORG 0000H           ; Initialize the memory
MOV R0,#34H          ; Register R0 loaded with first data location 34h
MOV R6,#00H          ; Load the register R6 with zero
MOV A,@R0            ; Read first number into accumulator
INC R0               ; Increment R0 (34H+1=35H)
MOV B,@R0            ; Read second number into register B
SUBB A,B              ; Subtraction of two numbers (A=A-B)
MOV R5,A              ; Move the acc content into R5 register
JNC EXIT             ; Jump to label EXIT if no carry generated
DEC R6               ; Decrement the register content (indicates negative number)
EXIT: SJMP EXIT       ; Specifies logical end of the program
END                  ; End of the program
```

7) Write an ALP to multiply two 8-bit numbers stored at 30h & 31h and store 16-bit result in 32h and 33h of internal RAM.

```
ORG 0000H      ; Initialize the memory
MOV R0,#30H    ; Load R0 with 30H (address of first number)
MOV A, @R0     ; Copy the first number into accumulator
INC R0         ; Increment R0( R0=31H Address of second number)
MOV B,@R0      ; Copy the second number into register B
MUL AB         ; Multiply the contents of accumulator and register B. Product is stored in A
               ; and B
INC R0         ; Increment R0
MOV @R0, A     ; Copy the contents of accumulator to 32H(Lower order byte of the product)
INC R0         ;Increment R0
MOV @R0,B      ; Copy the contents of register B to 33H(Higher order byte of the product)
END            ; End of the program
```

8) Write an ALP to divide two 8-bit numbers stored at 20h & 21h and store the quotient in 22H and remainder in 23H

```
ORG 0000H      ; Initialize the memory
MOV R0,#20H    ; Load R0 with 20H (address of first number)
MOV A, @R0     ; Copy the first number into accumulator
INC R0         ; Increment R0( R0=21H Address of second number)
MOV B,@R0      ; Copy the second number into register B
DIV AB         ; Divide the contents of accumulator by contents of register B. Quotient is
               ; stored in A and remainder in B.
INC R0         ; Increment R0 (R0=22H)
MOV @R0, A     ; Copy the contents of accumulator to 22H(Quotient)
INC R0         ;Increment R0 (R0=23H)
MOV @R0,B      ; Copy the contents of register B to 23H(Remainder)
END            ; End of the program
```

9) (a) Write an ALP to find if the number in 20H is positive or negative. If positive, store 00H and if negative, store FFH in 21H.

```
ORG 0000H    ; Initialize the memory
MOV A, 20H    ; Copy the number into accumulator
RLC A        ; Rotate the accumulator to the left through carry
JC NEGATIVE   ; Jump to NEGATIVE if CY=1
MOV 21H, #00H ; Copy 00H to 21H (Number is positive)
SJMP SKIP     ; Jump to SKIP
NEGATIVE: MOV 21H, #0FFH ; Copy FFH to 21H (Number is negative)
SKIP:      NOP          ; No operation
           END          ; End of the program
```

(b) Write an ALP to separate positive and negative number in a given array.

```
ORG 0000H    ; Initialize the memory
MOV R0, #30H  ; Starting address of the array
MOV R1, #50H  ; Address for storing positive numbers after separation
MOV R2, #05H  ; Array size is 5
BACK: MOV A, @R0 ; Copy the data from address location of R0 into Accumulator
JB ACC.7, SKIP ; If MSB bit of accumulator is 1 then jump to label SKIP
MOV @R1, A    ; If MSB bit of accumulator is Zero then move to address
               location of register R1
INC R1        ; Increment R1
SKIP: INC R0   ; Increment R0
DJNZ R2, BACK ; Decrement R2, if not zero then jump to label BACK
END           ; End of the program
```

10) (a) Write an ALP to find if the number in 20H is even or odd. If even, store 00H and if odd, store FFH in 21H.

```
ORG 0000H    ; Initialize the memory

MOV A, 20H    ; Copy the number into accumulator

RRC A        ; Rotate the accumulator to the right through carry

JC ODD        ; Jump to ODD if CY=1

MOV 21H, #00H ; Copy 00H to 21H (Number is even)

SJMP SKIP     ; Jump to SKIP

ODD :        MOV 21H, #0FFH ; Copy FFH to 21H (Number is odd)

SKIP:        NOP          ; No operation

END          ; End of the program
```

(b) Write an 8051 ALP to separate odd elements in a given array.

```
ORG 0000H    ; Initialize the memory

MOV R0, #30H  ; Starting address of array data

MOV R1, #50H  ; address for storing odd numbers after separation

MOV R2, #05H  ; array size 5

BACK: MOV A, @R0    ; copy data from address present in R0

JNB acc.0, SKIP    ; if LSB bit 0 of accumulator is not set jump to label skip

MOV @R1, A        ; if LSB bit 0 of acc is set, then move it to odd array

INC R1            ; Increment R1 by 1

SKIP: INC R0       ; Increment R0 by 1

DJNZ R2, BACK     ; Decrement R2 if not zero jump to label BACK

END              ; End of the program
```

(c) Write an 8051 ALP to separate even elements in a given array.

```
ORG 0000H ; Initialize the memory

MOV R0,#30H ; Starting address of array data

MOV R1,#50H ; address for storing even numbers after separation

MOV R2,#05 ; array size 5

BACK: MOV A,@R0 ; copy data from address present in R0

JB acc.0, SKIP ; if LSB bit 0 of accumulator is set jump to label skip

MOV @R1,A ; if LSB bit 0 of acc is set, then move it to even array

INC R1 ; Increment R1 by 1

SKIP: INC R0 ; Increment R0 by 1

DJNZ R2, BACK ; Decrement R2 if not zero jump to label BACK

END ; End of the program
```

11) (a) Write an ALP to arrange the 10 numbers starting at 20H in Ascending order.

```
ORG 000H          ; Initialize the memory

MOV R2, #09        ; Load R2 with 09 (Counter for outer loop)

AGAIN: MOV R1, #09   ; Load R1 with 09 (Counter for inner loop)

        MOV R0, #20H ; Load R0 with 20H (Starting address of the array)

BACK:   MOV A,@R0     ; Copy the contents of memory location pointed by R0 into acc.

        INC R0        ; Increment R0

        MOV B, @R0    ; Copy the contents of memory location pointed by R0 into B

        CJNE A,B,NEXT ; Compare the contents of acc with the contents of register B and
                        ; jump to NEXT if A is not equal to B

NEXT:   JC SKIP       ; Jump to SKIP if CY=1

        MOV @R0, A     ; Copy the contents of accumulator into memory location pointed
                        ; by R0

        DEC R0         ; Decrement R0

        MOV @R0, B     ; Copy the contents of register B into memory location pointed by
                        ; R0

        INC R0         ; Increment R0

SKIP:   DJNZ R1, BACK  ; Decrement R1 and jump to BACK if R1 not equal to zero

        DJNZ R2, AGAIN ; Decrement R2 and jump to AGAIN if R2 not equal to zero

END
```

(b) Write an ALP to arrange the 10 numbers starting at 20H in Descending order.

```
ORG 000H          ; Initialize the memory

MOV R2, #09        ; Load R2 with 09 (Counter for outer loop)

AGAIN: MOV R1, #09   ; Load R1 with 09 (Counter for inner loop)

        MOV R0, #20H ; Load R0 with 20H (Starting address of the array)

BACK:   MOV A,@R0     ; Copy the contents of memory location pointed by R0 into acc.

        INC R0        ; Increment R0

        MOV B, @R0    ; Copy the contents of memory location pointed by R0 into B

        CJNE A,B,NEXT ; Compare the contents of acc with the contents of register B and
                        ; jump to NEXT if A is not equal to B

NEXT:   JNC SKIP      ; Jump to SKIP if CY=0

        MOV @R0, A    ; Copy the contents of accumulator into memory location pointed
                        ; by R0

        DEC R0        ; Decrement R0

        MOV @R0, B    ; Copy the contents of register B into memory location pointed by
                        ; R0

        INC R0        ; Increment R0

SKIP:   DJNZ R1, BACK  ; Decrement R1 and jump to BACK if R1 not equal to zero

        DJNZ R2, AGAIN ; Decrement R2 and jump to AGAIN if R2 not equal to zero

END
```

12) (a) Write an ALP to find the smallest number in an array of 10 elements starting at 20H. Store the smallest number in 40H.

```
ORG 0000H      ; Initialize the memory
MOV R1, #09    ; Load R1 with 09 (Counter)
MOV R0, #20H   ; Load R0 with 20H (Starting address of the array)
MOV A, @R0     ; Copy the contents of memory location pointed by R0 into
               ; accumulator
BACK: INC R0    ; Increment R0
MOV B, @R0     ; Copy the contents of memory location pointed by R0 into register B
CJNE A, B, NEXT ; Compare the contents of accumulator with the contents of register B
               ; and jump to NEXT if A not equal to B
NEXT: JC SKIP  ; Jump to SKIP if CY=1
MOV A, B       ; Copy the contents of register B into accumulator
SKIP : DJNZ R1, BACK; Decrement R1 and jump to BACK if R1 is not equal to zero
MOV 40H, A     ; Copy the contents of accumulator (smallest number) to 40H
END            ;End of the program
```

(b) Write an ALP to find the largest number in an array of 10 elements starting at 20H. Store the largest number in 40H.

```
ORG 0000H      ; Initialize the memory
MOV R1, #09    ; Load R1 with 09 (Counter)
MOV R0, #20H   ; Load R0 with 20H (Starting address of the array)
MOV A, @R0     ; Copy the contents of memory location pointed by R0 into acc
BACK: INC R0    ; Increment R0
MOV B, @R0     ; Copy the contents of memory location pointed by R0 into register B
CJNE A, B, NEXT ; Compare the contents of accumulator with the contents of register B
               ; and jump to NEXT if A not equal to B
NEXT: JNC SKIP  ; Jump to SKIP if CY=0
MOV A, B       ; Copy the contents of register B into accumulator
SKIP : DJNZ R1, BACK; Decrement R1 and jump to BACK if R1 is not equal to zero
MOV 40H, A     ; Copy the contents of accumulator (smallest number) to 40H
END            ;End of the program
```